

COMPILER CONSTRUCTION - HANDWRITTEN REPORT

TITLE : PicoScript

THEME : 8-bit adventure

TEAM : Syeda Batool Kozmi

23K-0672

Rabia Zulfikar

23K-0851

① GAME STATE TRANSITION TABLE.

* Flag system

Trigger	Condition	State before	Action	State After
Flag IDENT = true	during world init	undefined	set flag	Flags[IDENT] = true
Flag IDENT = false	during world init	undefined	set flag	Flags[IDENT] = false
requires Flag X	Flag X = true	room locked	allow transition	room accessible
requires Flag X	Flag X = false	attempt action	block action	no change

* Player inventory system

Trigger	Condition	State before	Action	State After
inv ITEM	item exists in world	not in inventory pickup	Pick up item	inventory += [ITEM]
inv has ITEM	used in a condition	ITEM in inventory	allow condition	unchanged
use ITEM	ITEM is in inventory	item unused	run item use block	possible state change
use ITEM	ITEM is not in inventory	invalid	reject	unchanged

* Room Transitions

go north → ROOM-B	exit exists	in ROOM-A	move player	curr_room = ROOM-B
go direction → ROOM	requires flag / item satisfied	locked exit	allow move	room change
go direction	missing exit	any	invalid move	unchanged.

* Item interactions

Trigger	Condition	State before	Action	State after
examine ITEM	item exists	idle	show desc.	unchanged
use ITEM	block item has use	in inventory	execute statement	may modify flags/room/link
use ITEM	no use block	in inventory	no-op or error	unchanged

* Room entry events

on enter	entering room	room not yet processed	execute statements	may change flags/items/game state
----------	---------------	------------------------	--------------------	-----------------------------------

* Puzzle/Gate system

puzzle evaluated	requires condition true	gate locked	unlock target	room/door becomes accessible
puzzle evaluated	condition false	gate locked	nothing	unchanged
unlock IDENT	requirements met	locked entity	set unlocked	entity becomes usable

* NPC system

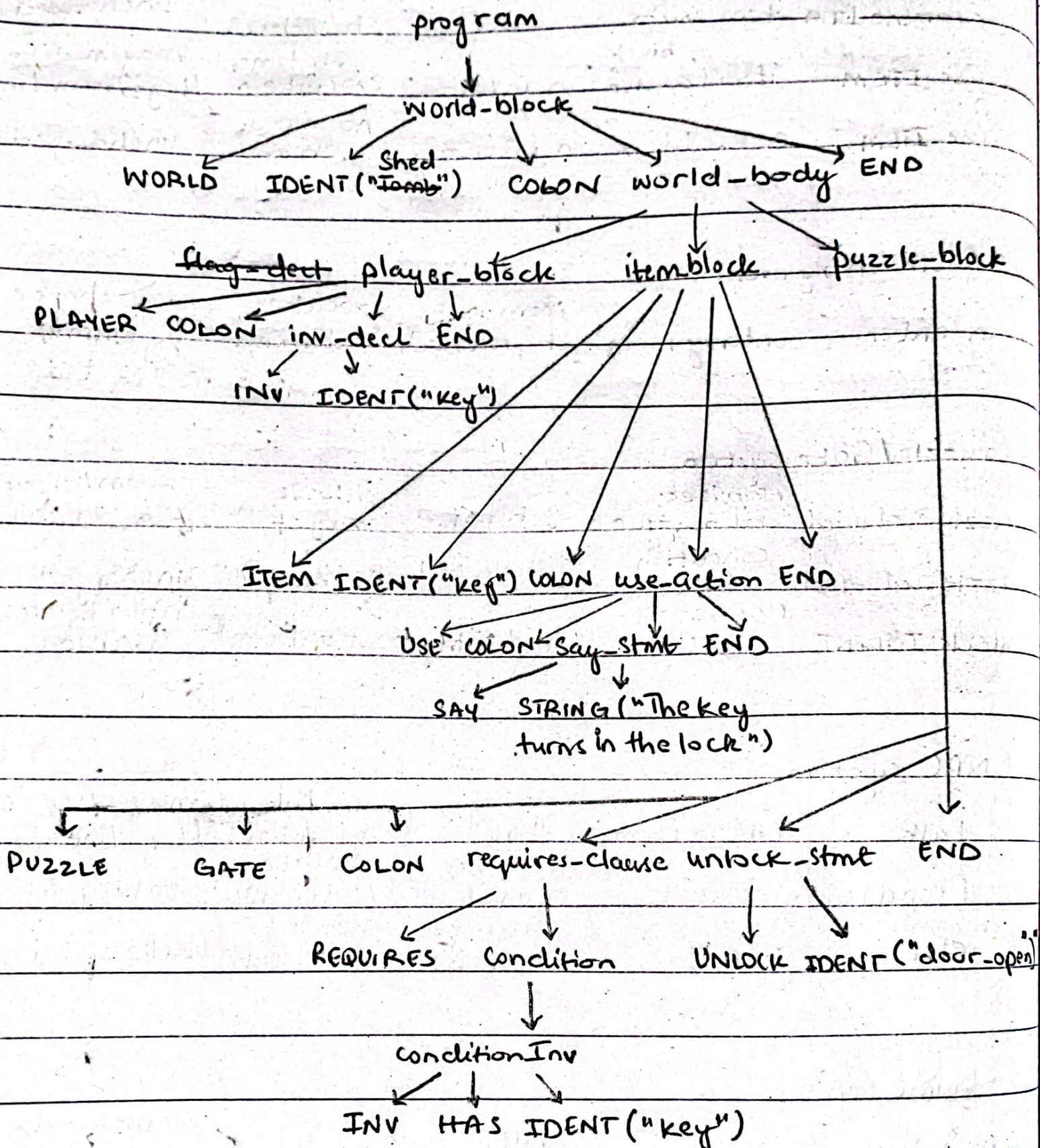
talk	NPC in room	dialogue start	run dialogue branches	may change flags/inventory
if condition	true	dialogue branch	execute statements	state updated
else	condition false	alternate branch	execute statements	state updated

* Game termination

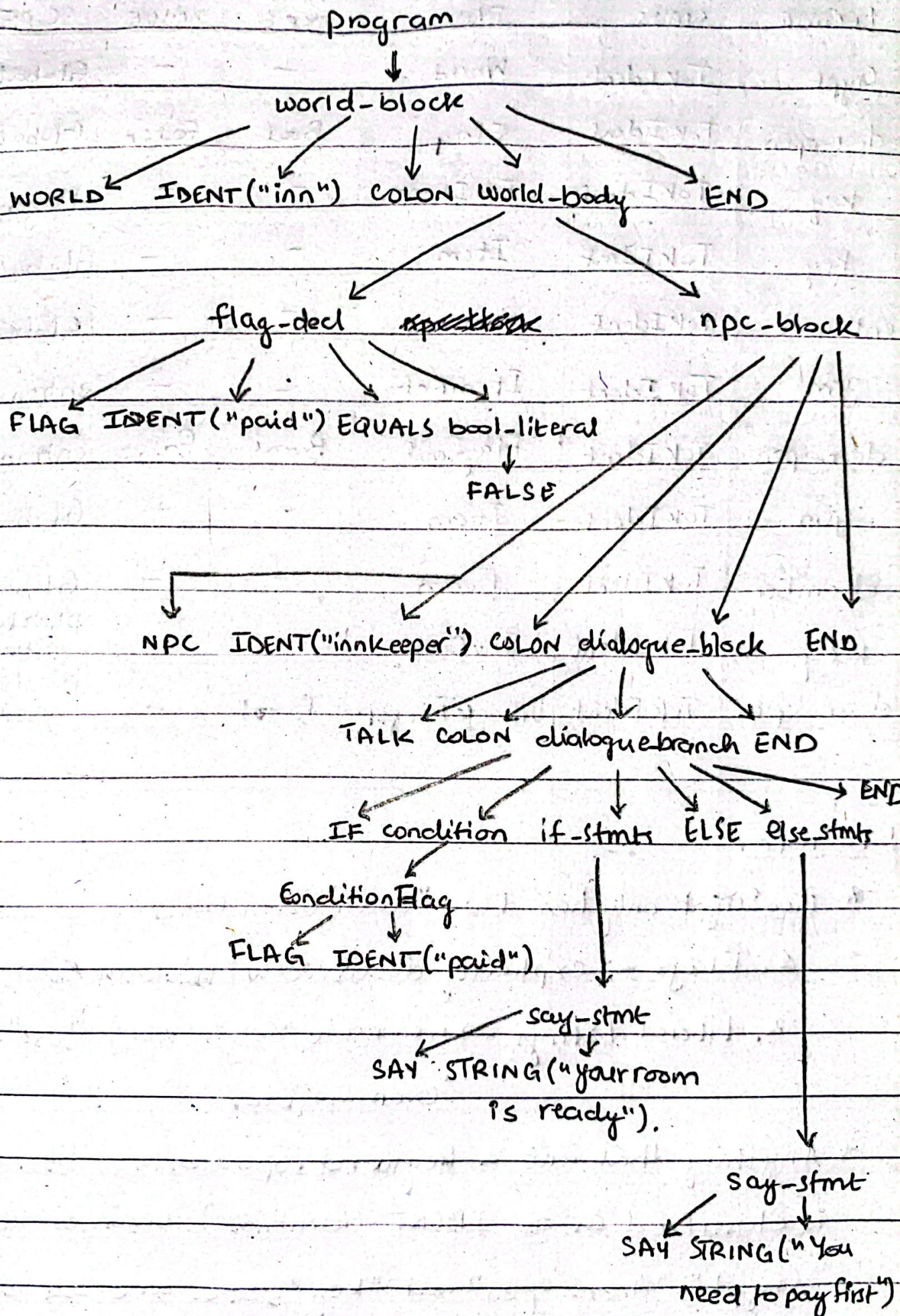
win STRING	any execution point	Playing	set win	game-status = won
lose STRING	any execution point	Playing	set lose	game-status = lost

② PARSE TREES

* Sample input: ~~tomb~~.pico



* Sample input : inn-pico



③ SYMBOL TABLE : example input : crypt.pico

LEXEME	TOKEN CLASS	SYMBOL TYPE	DATA TYPE	VALUE	SCOPE	LINE #
			-	-	Global	1
Crypt	TokIdent	World			Global	2
door-open	TokIdent	Flag	Bool	false	Global	4
key	TokIdent	InvItem	-	-	Player	4
key	TokIdent	Item	-	-	Global	6
entrance	TokIdent	Room	-	-	Global	9
gem	TokIdent	ItemRef	-	-	entrance	10
door-open	TokIdent	FlagCond	Bool	False	entrance	11
gem	TokIdent	Item	-	-	Global	13
chamber	TokIdent	Room	-	-	Global	18
key	TokIdent	InvCond	-	-	puzzle gate	25
door-open	TokIdent	UnlockTarget	Bool	-	puzzle gate	26

! To note:

* PicoScript only has two "data" definitions:

1. Flags → explicitly declared w/ boolean values

2. Literals/Strings → i.e. inside "Say", "win", "lose", "examine", etc.

* Anything that isn't a keyword, operator, or literal is classified as an IDENT (identifier). Lexemes like "Crypt", "door-open" and "key", etc. are all identifiers and NOT keywords.

* Bool value = "-" in the last entry. "Unlock" does not assign a literal, it flips flag to "true" at runtime.

The symbol table just doesn't capture the final value, only initials.